Specifications

Current intended behavior and current system structure.

Working draft

Current intended behavior and current system structure.

Document status

This index describes the repo as it exists in code today.

Use the docs this way:

- docs/specifications.md = current implemented behavior and current structure
- docs/implementation-guide.md = architecture, decisions, and canon-candidate direction
- docs/roadmap.md = planned work, sequencing, and open questions
- docs/concepts.md = shared term reference

This document should stay strict about current code truth. If a feature is visible but disabled, it should be described as disabled. If behavior is still planned, it belongs in the roadmap or implementation guide rather than here.

Current summary

The app is an Expo Router application with a public OWA site and a dedicated Nexus shell under /nexus/*.

Current live Nexus areas include:

- packet-backed discussions with shared fortress write preparation and finalize paths
- packet-backed trust and roles surfaces
- a read-only votes workspace
- a scoped Library browse surface
- a shell-level Packet Explorer overlay for deep inspection, lineage, grouped links, and action visibility

Current source split:

- core/* = portable packet logic, schemas, builders, contracts, and pure projections
- runtime/* = persistence, runtime services, and API glue

- app/* = application-layer components, hooks, constants, public content, and shared UI/state
- src/app/* = Expo Router route shell and API entrypoints

Chapters

- Product Surface
- Nexus Routes And Workflows
- Architecture And State
- Known Gaps And Provisional Notes

Public action/link model

- Public content actions are represented by PublicPageAction in app/public/content-types.ts.
- Actions may target internal public routes, external URLs, or static download paths under /downloads/.
- Existing internal route href usage remains supported as a compatibility bridge while newer content should prefer explicit target objects.

Public docs page structure

- The /docs route renders a public docs hero, document directory, selected readable document, and downloads/resources panel.
- The Charter, Nexus README, Implementation Guide, Specifications, and Roadmap are currently selectable and readable on-page through PublicDocumentReader and PUBLICREADABLEDOCUMENTS.
- Directory entries currently include the OWA Charter, Nexus README, Implementation Guide, Specifications, and Roadmap.
- Generated Markdown download actions point at /downloads/* artifacts and trigger browser downloads on web. PDF actions remain disabled placeholders until the PDF export pipeline is implemented.

Public docs generated artifacts

- The /docs page can switch its readable panel between Charter, Nexus README, Implementation Guide, Specifications, and Roadmap entries.
- Directory read buttons select the readable document in-page and scroll to the reader anchor; Markdown download buttons point at generated static files under /downloads/.
- scripts/build-public-docs.mjs is the current source of truth for compiling public Markdown sources into generated reader data and static Markdown downloads.
- Generated PDF actions remain disabled placeholders until a PDF export step is added.

- The document reader displays a local outline and anchored section cards for long generated documents; this is separate from the reusable secondary navigation system for now.

Public Docs Reader Behavior

- Public document cards may expose a Read Below action that selects the current document and scrolls the Docs page shell to the readable document panel.
- Reader outline items scroll within the public page shell rather than using browser-level document scrolling.
- Markdown downloads use static files generated into public/downloads/ and should be triggered as browser downloads on web.
- PDF actions remain disabled placeholders until the PDF pipeline is added.

Product Surface

Scope today

The product currently has two visible layers:

- a public OWA website shell for orientation and placeholder public pages
- a dedicated Nexus shell under /nexus/* for the first guest-facing OWA Nexus slice

Implemented surface today includes:

- public Home, About, and Docs pages inside the public shell
- Nexus identity routes for sign-in, create, claim, restore, and security
- a locality creation route for canonical geographic assembly creation
- Nexus routes for Dashboard, Discussions, Votes, Roles, Trust, and Library
- a shell-level Packet Explorer overlay
- a hidden wrapper-level /nexus/account custody route reached from the profile area

Not implemented today:

- remote multi-node sync beyond the local SQLite packet store
- real-time chat
- protected or private spaces
- trust-weighted ranking, moderation enforcement, delegation, or proposal execution
- dedicated routed packet detail pages outside Packet Explorer

Current surface roles

Library

Library is the scoped browse surface.

- it is packet-backed
- it defaults to a local-only view keyed to the active authority scope
- Open packet launches Packet Explorer without navigating away from Library
- packet actions on Library cards remain intentionally narrow

Packet Explorer

Packet Explorer is the inspect-and-traverse surface.

- it opens as a shell-level overlay
- it is still read-only for packet mutation, but its Home workspace now includes live search, import, and export workbenches
- it is separate from Library
- it supports session-persistent tabs with an auto-created Home tab
- its Home workspace now splits into Search, Import, and Export sub-tabs rendered directly in the shared inspector row
- Search is now a manual grouped packet finder over the current preferred packet index, with All, Direct, Name, and Text result groupings, capped preview slices inside All, and paged category views for deeper result review
- Import now supports two-step packet and bundle ingest through Analyze then Commit, with paste-first JSON input plus web-only .json file upload
- Export now supports raw preferred-revision packet export, bounded bundle export, full local-store bundle export, a compact direct packet lookup when no export target is preloaded, and an explicit cancel/reset path when a packet export target was preloaded earlier
- it now uses shared packet display-title normalization instead of exposing raw encoded packet ids as loaded titles where a human title can be derived
- it supports View as inspection lenses for Summary, Raw, Adapted, and Read Model
- it exposes Data, Lineage, Links, and Actions inspector rails using the same compact attached-tab style as the Home workspace tabs
- its packet tab deck now presents Home as the leftmost workspace tab, uses middle truncation for packet labels, and exposes desktop hover metadata for full packet titles and revision context
- its main content region now reads as one primary scroll surface inside the drawer instead of trapping long packet reads inside tiny inner scroll panes

- its packet-shell and inspector-control header bands can now collapse locally to reclaim vertical space without leaving helper rows behind, and collapsed bands are restored from the top command row
- it groups Incoming and Outgoing links by related packet, with explicit Open in new tab, Open in current tab, and View in Library actions for graph traversal
- it surfaces read-only NexusActionState and NexusActionIntentDescriptor data where available

Discussions

Discussions are the most mature interactive Nexus surface today.

- discussion feed, thread, and post workspaces are packet-backed
- reply, vote, expand, and create actions are now runtime-projected instead of being only page-local heuristics
- interactive writes use the shared fortress prepare/sign/finalize corridor
- compatibility still keeps legacy discussion family shapes readable while canonical discussion packets drive current writes

Votes

Votes are currently read-only.

- the route is packet-backed
- guests can inspect governance visibility cues and proposal or vote cards
- voting, objection, delegation, and execution workflows are not yet live
- visible disabled actions now use the same shared feature-status explainer pattern rather than acting like dead buttons

Current caveats

The following behaviors are not active product truth yet:

- initiative filtering in feeds, Library, or Explorer
- official versus unofficial packet visibility modes
- automoderation or visibility projection beyond current placeholders and packet-backed evidence reads
- policy-driven governance execution

Nexus Routes And Workflows

Public routes

- / = landing page for OWA
- /about = public explanation page
- /docs = public charter destination page and source-material explainer
- /login = redirect to /nexus/identity/sign-in
- /signup = redirect to /nexus/identity/create

Nexus routes

- /nexus redirects to /nexus/dashboard
- /nexus/dashboard = packet-backed dashboard
- /nexus/discussions = packet-backed discussion shell with Feed, Thread, and Post
- /nexus/votes = read-only vote floor
- /nexus/library = scoped packet browse surface
- /nexus/trust = scoped trust posture and relationship surface
- /nexus/roles = scope-centric role review surface
- /nexus/account = hidden wrapper-level custody route
- /nexus/identity/* = sign-in, create, claim, restore, and security ceremonies
- /nexus/locality/create = guided locality directory and creation flow

Core workflows

Nexus entry

1. User enters /nexus. 2. The route redirects to /nexus/dashboard. 3. The shell loads in Global Guest state. 4. The initial mounted baseline is Global + You.

Scope selection

1. User opens the scope menu or followed scopes. 2. The shell updates the active scope lens. 3. Dashboard, discussions, votes, roles, trust, and Library re-project against that scope.

Discussion posting

1. The route loads packet-backed discussion state. 2. The actor is a real Element(kind: "person"), including temporary guests. 3. The browser prepares a canonical mutation through the shared fortress corridor. 4. The active web identity shell signs the prepared packets locally. 5. Finalize re-verifies digest, signature, proof level, authority, and policy before persistence.

Trust and role review

1. Trust loads scope-local trust posture and relationship state. 2. Roles loads exact-scope role claims, claimants, and support or dispute evidence. 3. Protected guest actions open the shared auth-gate flow instead of trying raw writes.

Packet inspection

1. User opens Packet Explorer from the shell, Library, or link traversal. 2. Explorer loads packet summary, raw data, adapted data, read model, lineage, grouped links, and runtime action visibility. 3. Explorer stays read-only and uses session-backed tab state. 4. Library packet highlighting now clears itself when the highlighted packet is no longer represented by an Explorer packet tab.

Shell behavior

The Nexus shell currently provides:

- a dedicated Nexus layout separate from the public shell
- a two-column left-side navigation model on desktop
- a mobile overlay tray that opens the real menu rails immediately, uses Open menu as the visible trigger label, shrinks with rail collapse, and closes when both rails are minimized
- a desktop Packet Explorer drawer that keeps its width session-persistent and uses a dedicated drag-resize seam
- a session-scoped early-access welcome gate that blocks shell interaction until dismissed
- function-first versus scope-first as a shell preference
- You as a first-class personal scope lens
- independent rail collapse state
- a profile-area route into account and identity custody surfaces

Architecture And State

Active source split

- core/* holds portable packet logic, schemas, builders, interpreters, contracts, and pure projections
- runtime/* holds storage adapters, runtime services, query services, auth/trust/discussion orchestration, and API-facing glue

- app/* holds application-layer components, hooks, constants, public content, and shared shell state
- src/app/* holds the Expo Router route shell and API entrypoints

Current runtime and state patterns

- public pages are still mostly stateless
- Nexus state is shared through local React context providers
- shell and route projections load from packet-backed API routes
- the runtime store is SQLite-backed
- packet parsing runs through family compatibility and adaptation instead of route-local patches
- Packet Explorer session state is shell-level rather than route-level page state

Packet and compatibility foundations in active use

Current packet behavior in code includes:

- PacketEnvelope = { header, body }
- stable packet_id
- immutable revision_id
- multi-parent parentrevisionrefs
- family schema_version
- family revision_mode
- raw stored packets preserved as historical fact
- adapted runtime packets used as the normal read shape
- target-version-aware compatibility reads for supported families

Current runtime contracts worth keeping visible here:

- NexusActionState
- NexusActionIntentDescriptor
- NexusPacketExplorerPayload

Query and surface boundaries

- packet storage, compatibility, import/export, and merge remain below the route layer
- route components consume query payloads rather than storage classes directly
- runtime projects discussion and Explorer action visibility rather than leaving those rules entirely in page code

Current naming and structure notes

- route file names are lowercase and match path segments directly
- React components use PascalCase
- route screens continue to end with Page
- shared Nexus components live under app/components/nexus

The current repo is no longer using the older domain or storage root names. The active architecture is the core / runtime / app / src/app split described above.

Known Gaps And Provisional Notes

Current known gaps

- identity/auth and discussions remain the two strongest end-to-end verticals
- dashboard, votes, trust, roles, and Library are packet-backed, but are still comparatively provisional
- role creation and editing remain deferred
- packet-native follows remain deferred
- broader moderation, trust weighting, and governance execution remain deferred

Location and relationship caveats

- home_locality is now the packet-backed source of mounted geographic ancestry
- identity.location_disclosure remains optional profile metadata rather than mounted-scope truth
- assemblyassociation remains distinct from homelocality
- generic abstract assembly creation and richer locality governance are still later work

Explorer caveats

- Packet Explorer is read-only
- search, import/export, follow semantics, fork or adapt execution, diff, and compare are not live yet
- official versus unofficial initiative filtering is not active product behavior
- initiative-version subscription behavior is not active product behavior

Moderation and governance caveats

- moderation workflows are not yet implemented as live user-facing policy systems
- current disabled controls should be read as placeholders, not as hidden active features
- the first high-friction Nexus surfaces now attach compact inline explainers to disabled or partial controls so users can see whether a seam is coming soon, read-only, or partial
- proposals, votes, and decisions are present in packets and read surfaces, but governance is not yet a fully interactive trusted workflow

Provisional guidance

The following areas should still be treated as provisional:

- the shell-level early-access gate is now an honest entry warning, but it is not yet a tutorial, release-note system, or scope-aware onboarding flow
- the new feature-status registry and popover system currently covers Explorer, Votes, and Library first; broader surface rollout is still pending
- deeper trust weighting and moderation consequences
- vote execution, delegation, and propagation semantics
- long-term initiative filtering and visibility modes
- packet actions that currently appear as disabled placeholders
- any architecture or theory in the implementation guide that is not already represented in executable code